

popularity is that its syntax isn't cluttered with cryptic punctuation marks like other languages, making it easy to work with.

Explicit is better than implicit. If I'd written only "This is self-explanatory," I would have provided a terrible explanation for this aphorism. Similarly, in code, it's best to be verbose and explicit. You should avoid hiding code's functionality behind obscure language features that require deep language familiarity to fully understand.

Simple is better than complex. Complex is better than complicated. These two aphorisms remind us that we can build anything with simple or complex techniques. If you have a simple problem that requires a shovel, it's overkill to use a 50-ton hydraulic bulldozer. But for an enormous job, the complexity of operating a single bulldozer is preferable to the complications of coordinating a team of 100 shovelers. Prefer simplicity to complexity, but know the limits of simplicity.

Flat is better than nested. Programmers love to organize their code into categories, especially categories that contain subcategories that contain other sub-subcategories. These hierarchies often don't add organization so much as they add bureaucracy. It's okay to write code in just one top-level module or data structure. If your code looks like `spam.eggs.bacon.ham()` OR `spam['eggs']['bacon']['ham']`, you're making your code too complicated.

Sparse is better than dense. Programmers often like to cram as much functionality into as little code as possible, as in the following line:

```
print('\n'.join("%i bytes = %i bits which has %i possible values." % (j, j*8, 256**j-1) for j in (1 << i for i in range(8)))).
```

Although code like this might impress their friends, it'll infuriate their co-workers who have to try to understand it. Don't make your code try to do too much at once. Code that is spread out over multiple lines is often easier to read than dense one-liners. This aphorism is roughly the same as simple is better than complex.

Readability counts. Although `strcmp()` might obviously mean the "string compare" function to someone who has been programming in C since the 1970s, modern computers have enough memory to write out the full function name. Don't drop letters from your names or write